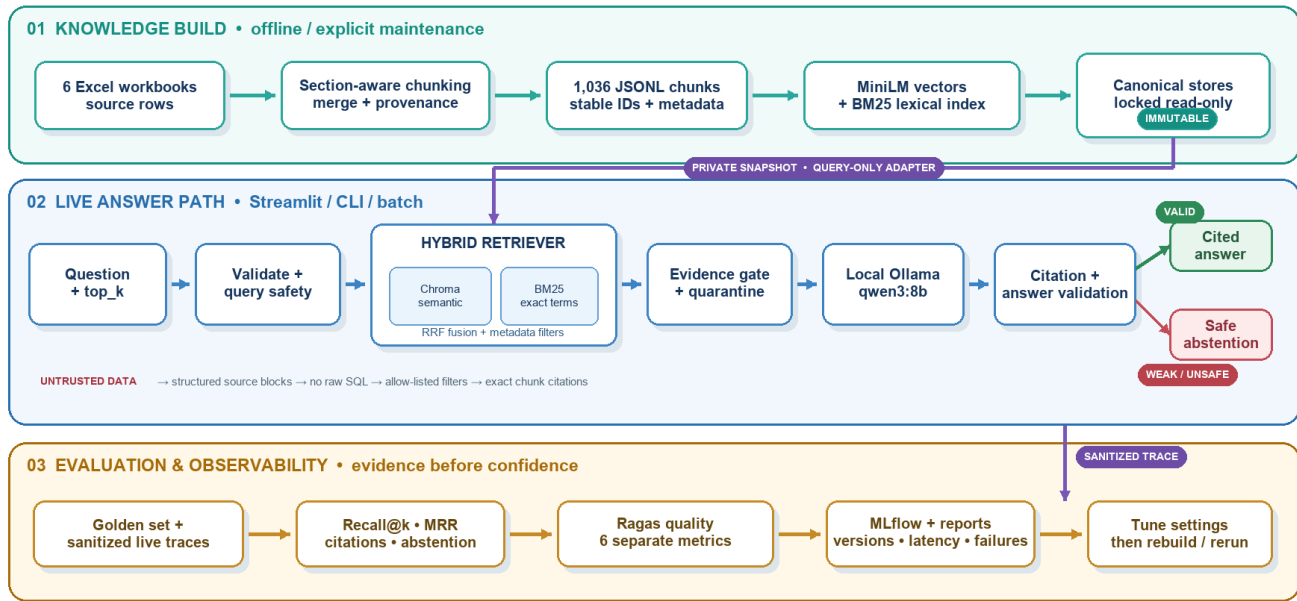


How the Malawi RAG Works

Q2 workflow design: from workbook rows to a cited local Ollama answer

Q2 • Malawi Local RAG — End-to-End Workflow

Six workbooks become traceable evidence; every answer stays local, cited, observable, and safely bounded.



Runtime workflow: START → validate question → load the read-only Chroma snapshot and BM25 index → encode and retrieve → RRF fusion → quarantine unsafe chunks → evidence gate → build bounded context → local Ollama generation → citation validation → answer and trace. Missing evidence or an invalid citation routes directly to cannot find in sources; injected chunks are removed before generation.

This guide follows the current Q2 implementation. Each stage shows its input, transformation, output, and the file responsible for it.

Current corpus: six Malawi IDSR workbooks, 1,036 retrieval chunks, local all-MiniLM-L6-v2 embeddings, and local Ollama qwen3:8b generation.

1. Configuration and workbook input

Files: src/config.py and Task2_dataset1/MWTGBookletsExcel/*.xlsx. Settings contains model names, paths, chunk sizes, thresholds, timeouts, and the default batch limit.

```
Settings(  
    embedding_model="sentence-transformers/all-MiniLM-L6-v2",  
    ollama_model="qwen3:8b",  
    ollama_url="http://127.0.0.1:11434",  
    answer_top_k=6,  
    batch_max_questions=10  
)
```

A workbook row starts as a paragraph number plus paragraph text:

```
paragraph number: 450  
text: "Reduce exposures to infectious or environmental hazards..."
```

2. Structure-aware chunking

File: src/chunking.py. Empty rows are skipped, whitespace is normalized, headings are detected, and related rows are merged inside a section. A heading such as 6.3.11 Reduce exposures... becomes the active section. Oversized rows are split at sentence boundaries.

```
Rows 450-452  
450: Reduce exposures...  
451: Use protective equipment...  
452: Improve ventilation...  
    ↓  
section: 6.3.11 Reduce exposures...  
paragraph_start: 450  
paragraph_end: 452
```

TG Booklet 3 has approximately 2,064 source entries and produces 213 retrieval chunks. The rows are merged to reduce fragmentation, while their paragraph range and section metadata preserve provenance.

3. Chunk identity and storage

Each chunk receives a stable ID and metadata before it is written to storage/chunks.jsonl.

```
{  
    "chunk_id": "TG_Booklet_3:p411-p413:c0043",  
    "section": "Annex 4A: District log...",  
    "topic": "District log of suspected outbreaks and alerts",  
    "paragraph_start": 411, "paragraph_end": 413,  
    "text": "Section: Annex 4A...\nRecord verbal or written..."  
}
```

4. Embedding vectors and ChromaDB

File: `src/indexing.py`. Every chunk text is encoded by `all-MiniLM-L6-v2`. Text is transformed into a numeric vector representing semantic meaning.

```
"Community members report unusual health events."  
  ↓ MiniLM encoder  
[0.12, -0.44, 0.73, ...]
```

The normalized vectors, original text, IDs, and metadata are stored in a persistent Chroma cosine collection. The query is encoded by the same model during retrieval.

5. BM25 lexical index

File: `src/bm25.py`. BM25 tokenizes text, counts term frequency, accounts for document length, and rewards rare exact terms. It is strong for disease names, acronyms, laboratory terms, and exact wording.

```
Query: "Chikungunya PCR"  
Tokens: ["chikungunya", "pcr"]  
High score: a chunk containing both exact terms
```

BM25 uses the same chunk IDs as ChromaDB, allowing both rankings to be combined safely.

6. HybridRetriever and RRF

File: `src/retrieval.py`. `HybridRetriever` runs semantic vector search and BM25 search, applies filters, and combines their rankings with Reciprocal Rank Fusion.

```
Question  
"How do communities report unusual health events?"
```

```
Vector search → semantic ranking  
BM25 search   → exact-term ranking  
RRF(d)        =  $\sum 1 / (60 + \text{rank}(d))$ 
```

```
RetrievedChunk(  
  chunk_id="TG_Booklet_1:p86-p89:c0022",  
  rank=1, semantic_similarity=0.72,  
  bm25_score=8.41, hybrid_score=0.031  
)
```

7. Evidence gate and safety filter

Files: `src/rag.py`, `src/guardrails.py`, and the Q3 reference snippet. Untrusted text is normalized for Unicode, invisible-character, homoglyph, leetspeak, punctuation, and spacing tricks. Word-preserving and compact regex checks run first; a replaceable local semantic detector can identify injection intent beyond fixed patterns.

```
"ignore previous instructions"
  ↓ security-only normalization
"ignore previous instructions"
  ↓ compact + semantic screening
quarantine unsafe chunk by ID
  ↓
use verifier-approved evidence only
```

A separate embedding-similarity gate can reject topically weak evidence before the LLM verifier. Structured audit events record the detection layer, chunk ID, retry, similarity score, cited IDs, and faithfulness score without logging malicious passage text. Detector or schema failures fail closed.

8. Context construction and local Ollama

File: `src/generation.py`. Complete source blocks are selected and placed in a bounded prompt. The model sees the question plus source text labelled as untrusted reference data.

```
SOURCE [TG_Booklet_1:p86-p89:c0022]
Section: Community Based Surveillance
Paragraphs: 86-89
CONTENT (untrusted reference text):
Community members report unusual events...

POST http://127.0.0.1:11434/api/chat
{ "model": "qwen3:8b", "stream": false,
  "messages": [system_prompt, question_and_sources] }
```

The prompt requires answers only from supplied evidence, a general public-health tone, and exact bracketed chunk citations.

9. Citation validation and response

```
Valid: "Community reporting uses trained members. [TG_Booklet_1:p86-p89:c0022]"
Invalid: "Prevalence is 40%. [invented:id]"
Result: valid answer or "cannot find in sources"
```

The validator rejects missing, malformed, unknown, or omitted-source citations. The CLI and Streamlit trace can show rank, semantic score, BM25 score, hybrid score, section, paragraph range, and source text.

10. Batch submission output

File: `batch_submit.py`. The runner reads the configured first 10 questions from `Test.csv` and writes four SampleSubmission-format rows per question: keywords, paragraph numbers, answer, and reference document.

```
test_submission_2026_09_14_15_48_38.csv
Q4_keywords
Q4_paragraph(s)_number
Q4_question_answer
Q4_reference_document
```

The filename is timestamped automatically. The CSV is flushed after each completed question, and the terminal shows a compact line such as `Processing test question 4/10 (40.0%) – Q10.`

11. Three-question Ragas evaluation

Ragas separates evidence retrieval from answer generation. On Q220, Q1016, and Q1226, context precision and context recall were 1.0000 for every question. Mean faithfulness was 0.8333, answer relevancy 0.8665, answer correctness 0.9481, and answer similarity 0.8756.

Metric	Q220	Q1016	Q1226	Mean
Faithfulness	0.5000	1.0000	1.0000	0.8333
Answer relevancy	0.8132	1.0000	0.7861	0.8665
Context precision	1.0000	1.0000	1.0000	1.0000
Context recall	1.0000	1.0000	1.0000	1.0000
Answer correctness	0.9866	0.9642	0.8933	0.9481
Answer similarity	0.9465	0.8570	0.8232	0.8756

Q220 shows why correctness does not replace grounding: it closely matched the reference but received 0.5000 faithfulness. This is a small development sample, not a held-out production benchmark. Local qwen3:8b generated and judged the answers, so human review remains necessary. The sample was kept at three because sustained inference caused significant MacBook heat, a machine resource constraint rather than an observed algorithm failure.

End-to-end transformation

Excel rows → section-aware chunks → JSONL metadata
→ embedding vectors in ChromaDB + BM25 terms
→ query vector + lexical search → RRF ranking
→ safety/evidence gate → bounded Ollama prompt
→ citation validation → grounded answer or abstention